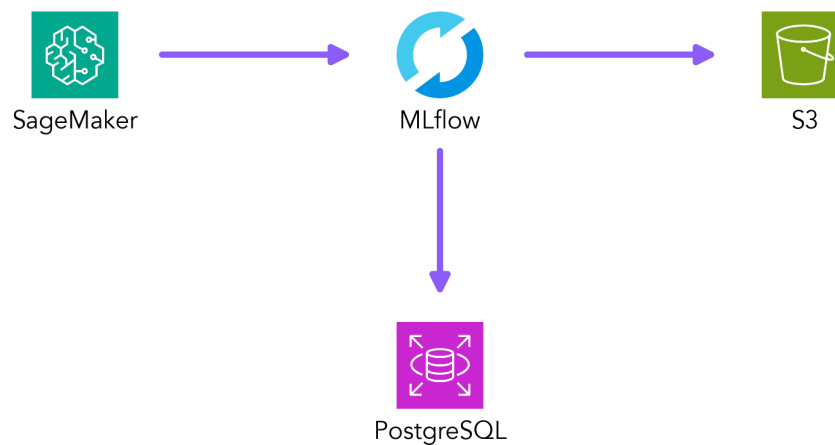




Credit Risk Modeling on AWS

MLflow Tracking Server with SageMaker

MLflow on AWS Architecture



What This Guide Covers:

- Deploy MLflow Tracking Server using AWS SAM
- Configure IAM roles for SageMaker Studio
- Train models with SageMaker and MLflow
- Store artifacts in S3 with PostgreSQL
- Use LocalStack for local development





Overview

This chapter demonstrates how to deploy MLflow Tracking Server infrastructure for Amazon SageMaker Studio using AWS SAM (Serverless Application Model). MLflow provides experiment tracking, model versioning, and artifact storage capabilities essential for production ML workflows. The architecture uses AWS SAM to deploy IAM roles and policies, enabling MLflow integration with SageMaker Studio for centralized experiment management.

MLflow was first integrated with Amazon SageMaker in June 2024 and is now the recommended replacement for SageMaker Experiments. In 2025, AWS announced MLflow Apps as the default tracking experience for SageMaker Studio, providing a serverless tracking experience. Since MLflow Apps are difficult to replicate locally, this chapter focuses on the classic MLflow Tracking Server deployment pattern that works nearly identically in local development and production.

For local development, we use Docker Compose to run LocalStack (for CloudFormation and IAM), PostgreSQL (for MLflow metadata), and MLflow Tracking Server (for experiment tracking). SageMaker Local Mode enables training models in Docker containers that log metrics and artifacts to the local MLflow server, providing a complete development environment without AWS costs.

Key Learning Objectives

By the end of this guide, you will understand:

- How to deploy MLflow infrastructure using AWS SAM templates
- IAM role configuration for MLflow Tracking Server and SageMaker integration
- Training models with SageMaker Local Mode and MLflow experiment tracking
- Artifact storage in S3 with PostgreSQL backend for metadata
- Local development workflow using LocalStack and Docker Compose

Why MLflow?

MLflow is an open-source platform for managing the ML lifecycle, including experimentation, reproducibility, and deployment. It provides four main components:

- **MLflow Tracking:** Record and query experiments (parameters, metrics, artifacts)
- **MLflow Projects:** Package ML code in a reusable, reproducible format
- **MLflow Models:** Deploy models in diverse serving environments
- **Model Registry:** Store, annotate, and manage models in a central repository



MLflow in SageMaker Studio

Amazon SageMaker has progressively integrated MLflow as the primary experiment tracking solution:

- **June 2024:** MLflow Tracking Server support added to SageMaker Studio
- **2025:** MLflow Apps announced as the default tracking experience
- **S3 Backend:** SageMaker MLflow uses S3 for artifact storage (not filesystem)
- **Serverless:** MLflow Apps provide managed, serverless tracking (likely ECS-based)

This chapter focuses on the classic deployment pattern using SAM, which provides more control and works consistently across local and production environments.

Architecture Components

The MLflow infrastructure consists of AWS SAM-deployed IAM resources and a local Docker Compose stack for development. The SAM template creates IAM roles and policies required for SageMaker Studio to use MLflow Tracking Server.

AWS SAM Template

The SAM template (`template.yaml`) deploys the following resources:

SAM-Deployed Resources

1. **MLflowTrackingServerRole:** IAM role for MLflow Tracking Server
 - Trust policy allows `sagemaker.amazonaws.com` to assume the role
 - Includes `AmazonSageMakerFullAccess` managed policy
 - Custom policy for S3 artifact access and CloudWatch logging
2. **SageMakerMLflowPolicy:** IAM policy attached to existing SageMaker execution role
 - MLflow Tracking Server management permissions (Create, Update, Delete, Start, Stop)
 - SageMaker Studio permissions for session management
 - IAM PassRole permission for MLflow role
 - S3 and CloudWatch Logs permissions

Local Development Stack

The Docker Compose configuration (`docker-compose.yml`) provides a complete local development environment:



Docker Compose Services

1. **LocalStack** (port 4566): Emulates CloudFormation, IAM, and S3 services
2. **PostgreSQL** (port 5433): Backend store for MLflow metadata (runs, experiments, metrics)
3. **MLflow Tracking Server** (port 5001): MLflow UI and API for experiment tracking

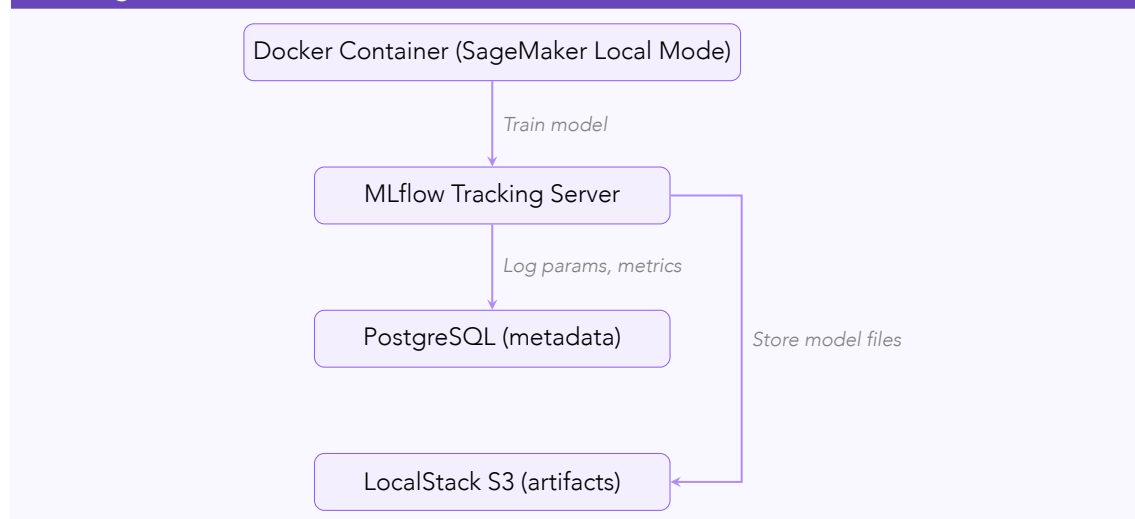
Service Configuration:

- MLflow artifacts are stored in LocalStack S3 (`s3://mlflow`)
- PostgreSQL stores experiment metadata (runs, parameters, metrics)
- Health checks ensure proper startup order
- Network bridge connects all services

SageMaker Local Mode with MLflow

SageMaker Local Mode allows training models in Docker containers on your local machine. Combined with MLflow, this provides a complete development workflow that mirrors production without AWS costs.

Training Workflow



Training Configuration

The training script runs inside a SageMaker container and connects to the host MLflow server:

- **MLflow URI:** `http://host.docker.internal:5001` (from container to host)
- **S3 Endpoint:** `http://host.docker.internal:4566` (for artifact storage)
- **Experiment:** Named experiment for organizing runs
- **Tracking:** Parameters, metrics, tags, and model artifacts



Model Logging APIs

MLflow API Considerations

MLflow provides two approaches for storing model artifacts:

- **log_artifact()**: Stores any file (plots, binaries, config). Simple and reliable, works without REST API configuration.
- **log_model()**: Flavor-specific model logging (sklearn, CatBoost, PyTorch). Requires MLflow 3.0+ with logged-models API support. More convenient for deployment workflows.

This implementation uses `mlflow.sklearn.log_model()` with MLflow 3.0+ which supports the logged-models API for better model tracking and deployment integration.

IAM Configuration

The SAM template creates IAM resources that enable SageMaker Studio to manage MLflow Tracking Servers. Understanding these permissions is essential for production deployments.

MLflow Tracking Server Role

This role is assumed by the MLflow Tracking Server when running in SageMaker:

```
# From template.yaml - MLflowTrackingServerRole
MLflowTrackingServerRole:
  Type: AWS::IAM::Role
  Properties:
    RoleName: !Sub 'MLflowTrackingServerRole-${StackCode}'
    AssumeRolePolicyDocument:
      Version: '2012-10-17'
      Statement:
        - Effect: Allow
          Principal:
            Service: sagemaker.amazonaws.com
          Action: sts:AssumeRole
    ManagedPolicyArns:
      - arn:aws:iam::aws:policy/AmazonSageMakerFullAccess
    Policies:
      - PolicyName: MLflowS3Access
        PolicyDocument:
          Version: '2012-10-17'
          Statement:
            - Effect: Allow
              Action:
                - s3:GetObject
                - s3:PutObject
                - s3:DeleteObject
                - s3:ListBucket
                - s3:GetBucketLocation
              Resource:
                - !Sub 'arn:aws:s3:::${SageMakerBucket}'
                - !Sub 'arn:aws:s3:::${SageMakerBucket}/${MLflowPrefix}/*'
            - Effect: Allow
```



```
Action:
- logs:CreateLogGroup
- logs:CreateLogStream
- logs:PutLogEvents
- logs:DescribeLogStreams
Resource:
- !Sub 'arn:aws:logs:${AWS::Region}:${AWS::AccountId}:...'
```

SageMaker Execution Role Policy

This policy is attached to an existing SageMaker execution role to enable MLflow management:

```
# Key permissions from SageMakerMLflowPolicy
{
  "Sid": "MLflowTrackingServerPermissions",
  "Effect": "Allow",
  "Action": [
    "sagemaker:CreateMlflowTrackingServer",
    "sagemaker:UpdateMlflowTrackingServer",
    "sagemaker>DeleteMlflowTrackingServer",
    "sagemaker:DescribeMlflowTrackingServer",
    "sagemaker:ListMlflowTrackingServers",
    "sagemaker:StartMlflowTrackingServer",
    "sagemaker:StopMlflowTrackingServer"
  ],
  "Resource": ["arn:aws:sagemaker:${Region}:${AccountId}:mlflow-tracking-server/*"]
}
```

Code Examples

Docker Compose Configuration

The complete Docker Compose configuration for local development:

```
services:
  localstack:
    image: localstack/localstack:latest
    container_name: mlflow-on-aws-localstack
    ports:
      - "4566:4566"
    environment:
      - AWS_DEFAULT_REGION=us-east-1
      - DATA_DIR=/var/lib/localstack
      - SERVICES=cloudformation,iam,s3
    volumes:
      - ./localstack_data:/var/lib/localstack
      - /var/run/docker.sock:/var/run/docker.sock
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:4566/_localstack/health"]
      interval: 10s
```



```
    timeout: 5s
    retries: 5

postgres:
  image: postgres:15
  container_name: mlflow-on-aws-postgres
  ports:
    - "5433:5432" # Use 5433 to avoid conflicts
  environment:
    - POSTGRES_USER=mlflow
    - POSTGRES_PASSWORD=mlflow
    - POSTGRES_DB=mlflow
  healthcheck:
    test: ["CMD-SHELL", "pg_isready -U mlflow"]
    interval: 5s
    timeout: 5s
    retries: 5

mlflow:
  build:
    context: .
    dockerfile: Dockerfile.mlflow
  image: mlflow-on-aws:latest
  container_name: mlflow-on-aws-server
  ports:
    - "5001:5000"
  environment:
    - AWS_ACCESS_KEY_ID=test
    - AWS_SECRET_ACCESS_KEY=test
    - MLFLOW_S3_ENDPOINT_URL=http://localstack:4566
    - MLFLOW_S3_IGNORE_TLS=true
    - AWS_DEFAULT_REGION=us-east-1
  command: >
    mlflow server
    --backend-store-uri postgresql://mlflow:mlflow@postgres:5432/mlflow
    --host 0.0.0.0
    --port 5000
    --serve-artifacts
    --artifacts-destination s3://mlflow
  depends_on:
    postgres:
      condition: service_healthy
    localstack:
      condition: service_healthy
```

Training Script

The training script that runs inside the SageMaker container:

```
"""Training script for SageMaker Local Mode with MLflow tracking."""
import argparse
import logging
import os
import joblib
import mlflow
import mlflow.sklearn
import pandas as pd
```



```
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import log_loss, roc_auc_score
from sklearn.model_selection import train_test_split

logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)

def train():
    """Training function that runs in SageMaker container."""
    parser = argparse.ArgumentParser()
    parser.add_argument("--mlflow-tracking-uri", type=str, required=True)
    parser.add_argument("--mlflow-experiment", type=str, required=True)
    parser.add_argument("--n-estimators", type=int, default=100)
    parser.add_argument("--max-depth", type=int, default=10)
    parser.add_argument("--train", type=str, default=os.environ.get("SM_CHANNEL_TRAIN"))
    parser.add_argument("--model-dir", type=str, default=os.environ.get("SM_MODEL_DIR"))
    args = parser.parse_args()

    # Configure MLflow S3 for LocalStack
    os.environ.setdefault("MLFLOW_S3_IGNORE_TLS", "true")
    s3_endpoint = os.environ.get("MLFLOW_S3_ENDPOINT_URL")
    if s3_endpoint and s3_endpoint.startswith("http"):
        os.environ["AWS_ENDPOINT_URL_S3"] = s3_endpoint

    # Configure MLflow
    mlflow.set_tracking_uri(args.mlflow_tracking_uri)
    mlflow.set_experiment(args.mlflow_experiment)
    logger.info(f"MLflow Tracking URI: {args.mlflow_tracking_uri}")
    logger.info(f"MLflow Experiment: {args.mlflow_experiment}")

    # Load dataset
    from pathlib import Path
    data_file = Path(args.train) / "heloc_dataset_v1.csv"
    df = pd.read_csv(data_file)
    logger.info(f"Loaded dataset: {df.shape}")

    # Prepare data
    label = "RiskPerformance"
    df[label] = df[label].map({"Good": 0, "Bad": 1})
    features = [col for col in df.columns if col != label]
    X = df[features]
    y = df[label]
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, random_state=42, stratify=y
    )

    # Handle missing values (-8 is missing value indicator)
    X_train = X_train.replace(-8, 0).fillna(0)
    X_test = X_test.replace(-8, 0).fillna(0)

    # Train with MLflow tracking
    with mlflow.start_run():
        mlflow.set_tag("model_type", "RandomForestClassifier")
        mlflow.log_param("n_estimators", args.n_estimators)
        mlflow.log_param("max_depth", args.max_depth)
        mlflow.log_param("model_type", "RandomForest")

    model = RandomForestClassifier(
        n_estimators=args.n_estimators,
        max_depth=args.max_depth,
        random_state=42,
```




```
        n_jobs=-1,
    )
    model.fit(X_train, y_train)

    # Evaluate
    test_pred = model.predict_proba(X_test)[:, 1]
    test_gini = roc_auc_score(y_test, test_pred) * 2 - 1
    test_log_loss = log_loss(y_test, test_pred)

    mlflow.log_metric("test_gini", test_gini)
    mlflow.log_metric("test_log_loss", test_log_loss)

    # Log model using mlflow.sklearn.log_model()
    try:
        mlflow.sklearn.log_model(model, "model")
        logger.info("Model logged to MLflow using logged-models API")
    except Exception as e:
        logger.warning(f"Failed to log model to MLflow: {e}")

    logger.info(f"Test Gini: {test_gini:.4f}")
    logger.info("Training complete!")

    # Save model for SageMaker
    import tarfile
    model_joblib_path = os.path.join(args.model_dir, "model.joblib")
    joblib.dump(model, model_joblib_path)

    # Package as tar.gz for SageMaker deployment
    model_tar_path = os.path.join(args.model_dir, "model.tar.gz")
    with tarfile.open(model_tar_path, "w:gz") as tar:
        tar.add(model_joblib_path, arcname="model.joblib")
    logger.info(f"Model packaged: {model_tar_path}")

if __name__ == "__main__":
    train()
```

SageMaker Local Mode Training

The main training script that configures SageMaker Local Mode and MLflow:

```
"""Train model using SageMaker Local Mode with MLflow tracking"""
import logging
from pathlib import Path
import boto3
from botocore.exceptions import ClientError
from sagemaker.estimator import Estimator
from sagemaker.local import LocalSession
from scripts.docker_utils import DockerManager

logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)

# Configuration
REGION = "us-east-1"
STACK_NAME = "mlflow-stack"
LOCALSTACK_ENDPOINT = "http://localhost:4566"
MLFLOW_TRACKING_URI = "http://host.docker.internal:5001" # From container
```



```
LOCALSTACK_S3_ENDPOINT = "http://host.docker.internal:4566"

def get_stack_role_arn() -> str:
    """Get MLflow Tracking Server role ARN from CloudFormation stack"""
    try:
        import urllib.request
        urllib.request.urlopen(f"{LOCALSTACK_ENDPOINT}/_localstack/health", timeout=2)
        endpoint_url = LOCALSTACK_ENDPOINT
        logger.info(f"LocalStack detected: {endpoint_url}")
    except Exception:
        endpoint_url = None
        logger.info("Using real AWS...")

    cfn = boto3.client("cloudformation", endpoint_url=endpoint_url, region_name=REGION)
    response = cfn.describe_stacks(StackName=STACK_NAME)
    stack = response["Stacks"][0]

    for output in stack.get("Outputs", []):
        if output["OutputKey"] == "MLflowTrackingServerRoleArn":
            return output["OutputValue"]

    raise RuntimeError(f"Role ARN not found in stack '{STACK_NAME}'")

def build_docker_image() -> str:
    """Build Docker image for SageMaker training with MLflow"""
    dockerfile_path = Path(__file__).parent / "training" / "Dockerfile.train"
    image_name = "mlflow-sagemaker-train:latest"
    docker_mgr = DockerManager()
    return docker_mgr.build_image(
        image_name=image_name,
        dockerfile_path=dockerfile_path,
        build_context=Path(__file__).parent,
    )

def train_with_sagemaker():
    """Train using SageMaker Local Mode with MLflow"""
    logger.info("Training with SageMaker Local Mode + MLflow")

    # Build Docker image
    image_name = build_docker_image()

    # Setup SageMaker Local Session
    sagemaker_session = LocalSession()
    sagemaker_session.config = {"local": {"local_code": True}}

    # Find data directory
    data_file = Path(__file__).parent.parent / "data" / "heloc_dataset_v1.csv"
    train_data_local = f"file://{data_file.parent}"
    output_path_local = f"file://{Path(__file__).parent / 'model_output'}"

    # Get IAM role from stack
    try:
        iam_role = get_stack_role_arn()
    except RuntimeError as e:
        logger.warning(f"Could not get role from stack: {e}")
        iam_role = "arn:aws:iam::111111111111:role/DummyRole"

    # Create estimator with MLflow configuration
    training_dir = Path(__file__).parent / "training"
    estimator = Estimator(
        image_uri=image_name,
```



```
role=iam_role,
instance_type="local",
instance_count=1,
sagemaker_session=sagemaker_session,
output_path=output_path_local,
source_dir=str(training_dir),
hyperparameters={
    "mlflow-tracking-uri": MLFLOW_TRACKING_URI,
    "mlflow-experiment": "credit-scoring-sagemaker",
    "n-estimators": 100,
    "max-depth": 10,
},
entry_point="train.py",
environment={
    "AWS_ACCESS_KEY_ID": "test",
    "AWS_SECRET_ACCESS_KEY": "test",
    "MLFLOW_S3_ENDPOINT_URL": LOCALSTACK_S3_ENDPOINT,
    "MLFLOW_S3_IGNORE_TLS": "true",
},
)

logger.info(f"Training data: {train_data_local}")
logger.info(f"MLflow URI: {MLFLOW_TRACKING_URI}")

# Train
estimator.fit({"train": train_data_local})
logger.info("Training complete!")
logger.info("MLflow UI: http://localhost:5001")

if __name__ == "__main__":
    train_with_sagemaker()
```

LocalStack S3 Setup

Script to create the S3 bucket in LocalStack for MLflow artifacts:

```
"""Setup S3 bucket in LocalStack for MLflow artifacts"""
import os
import time
import boto3
import requests
from botocore.config import Config
from botocore.exceptions import ClientError

# Configuration
LOCALSTACK_ENDPOINT = os.environ.get("LOCALSTACK_ENDPOINT", "http://localhost:4566")
MLFLOW_BUCKET = os.environ.get("MLFLOW_BUCKET", "mlflow")
REGION = os.environ.get("AWS_DEFAULT_REGION", "us-east-1")

def wait_for_localstack(max_retries: int = 30, delay: int = 2) -> None:
    """Wait for LocalStack to be ready."""
    print(f"Waiting for LocalStack at {LOCALSTACK_ENDPOINT}...")
    for i in range(max_retries):
        try:
            response = requests.get(
                f"{LOCALSTACK_ENDPOINT}/_localstack/health", timeout=2
            )
        except ClientError:
```



```
        if response.status_code == 200:
            print("LocalStack is ready")
            return
        except Exception:
            pass
        if i < max_retries - 1:
            print(f"Waiting... ({i + 1}/{max_retries})")
            time.sleep(delay)
        raise RuntimeError(f"LocalStack not ready after {max_retries * delay}s")

def setup_s3_bucket() -> None:
    """Create S3 bucket in LocalStack if it doesn't exist."""
    print(f"Setting up LocalStack S3 bucket: {MLFLOW_BUCKET}")
    wait_for_localstack()

    boto_config = Config(
        region_name=REGION,
        s3={"endpoint_url": LOCALSTACK_ENDPOINT, "addressing_style": "path"},
    )

    s3 = boto3.client(
        "s3",
        endpoint_url=LOCALSTACK_ENDPOINT,
        aws_access_key_id="test",
        aws_secret_access_key="test",
        region_name=REGION,
        config=boto_config,
    )

    try:
        s3.head_bucket(Bucket=MLFLOW_BUCKET)
        print(f"Bucket '{MLFLOW_BUCKET}' already exists")
    except ClientError as e:
        if e.response.get("Error", {}).get("Code") == "404":
            print(f"Creating bucket: {MLFLOW_BUCKET}")
            if REGION == "us-east-1":
                s3.create_bucket(
                    Bucket=MLFLOW_BUCKET,
                    CreateBucketConfiguration={"LocationConstraint": ""},
                )
            else:
                s3.create_bucket(
                    Bucket=MLFLOW_BUCKET,
                    CreateBucketConfiguration={"LocationConstraint": REGION},
                )
            print(f"Created bucket: {MLFLOW_BUCKET}")
        else:
            raise

    print("LocalStack S3 setup complete!")

if __name__ == "__main__":
    setup_s3_bucket()
```



Deployment Workflow

Local Development

```
# Start all services (LocalStack, PostgreSQL, MLflow)
make start

# Build SAM application
make build

# Deploy stack to LocalStack
make deploy-stack

# Train model with SageMaker Local Mode + MLflow
make train

# Access MLflow UI
open http://localhost:5001
```

Production Deployment

```
# Deploy to real AWS (requires AWS credentials)
AWS_DEPLOY=true make deploy-stack

# Verify stack outputs
make describe-stack

# Stack outputs include:
# - MLflowTrackingServerRoleArn: Role for MLflow Tracking Server
# - MLflowTrackingServerRoleName: Role name to use in SageMaker Studio
# - MLflowArtifactsPath: S3 path for artifacts (s3://bucket/mlflow)
```

SageMaker Studio Integration

After deploying the stack to AWS:

1. Navigate to SageMaker Studio
2. Create a new MLflow Tracking Server
3. Select role: `MLflowTrackingServerRole-{StackCode}`
4. Configure artifact store: `s3://{bucket}/{prefix}`
5. Wait for IAM propagation (1-2 minutes)
6. Start using MLflow for experiment tracking



Key Takeaways

What You've Learned

- **AWS SAM:** Deploy MLflow infrastructure using serverless templates
 - IAM roles for MLflow Tracking Server
 - Policies for SageMaker execution role integration
 - CloudFormation stack management
- **MLflow Tracking:** Experiment management and model versioning
 - PostgreSQL backend for metadata storage
 - S3 artifact storage for models and plots
 - `log_model()` API for model deployment workflows
- **SageMaker Local Mode:** Train models locally with production-like environment
- **LocalStack:** Complete local AWS emulation for development without costs
- **Docker Compose:** Orchestrate multi-service development environment

Further Reading

Additional Resources

For more information about MLflow and AWS SAM:

- **GitHub Repository:** github.com/deburky/credit-risk-modeling-on-aws
- **AWS SAM Documentation:** docs.aws.amazon.com/serverless-application-model
- **MLflow on SageMaker:** docs.aws.amazon.com/sagemaker/latest/dg/mlflow.html
- **MLflow Documentation:** mlflow.org/docs/latest